Report

Introduction

Accurate localization and mapping are foundational capabilities for autonomous mobile robots operating in unknown environments. A robot must estimate its own pose while simultaneously reconstructing a consistent representation of the surrounding space.

In this project, I implement a multi-sensor SLAM pipeline on a differential-drive robot using encoder and IMU odometry, 2-D LiDAR scans, and RGB-D measurements. Wheel encoders and the IMU provide high-rate motion estimates but suffer from cumulative drift. LiDAR measurements offer geometric constraints that enable scan alignment and map consistency. RGB-D data provides dense depth and color information, allowing the geometric map to be augmented with visual appearance.

The primary objective is twofold:

1. **Robot Localization and Geometric Mapping:**
   Fuse odometry and 2-D LiDAR measurements to estimate the robot trajectory and construct a consistent 2-D occupancy grid map of the environment.
2. **Appearance Augmentation via RGB-D:**
   Use RGB-D images to project color information onto the 2-D floor map, producing a visually enriched representation of the environment.

Problem Formulation&Technical Approach

Let the robot pose at time t be

$$\mathbf{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} \in \mathrm{SE}(2),$$

where $(x_t, y_t) \in \mathbb{R}^2$ is the robot position in the world frame and $\theta_t \in \mathbb{R}$ is its heading.
At each time step, we are given asynchronous sensor measurements:
Odometry measurements:

$$\mathbf{u}_t = (v_t, \omega_t)$$

where v_t is linear velocity derived from encoder counts and ω_t is yaw rate from the IMU.
LiDAR scan:

$$\mathcal{Z}_t = \{r_{t,i}, \phi_i\}_{i=1}^{N}$$

consisting of range measurements $r_{t,i}$ at known angles ϕ_i.
Convert r, ϕ to x, y

$$\mathbf{p}_{t,i} = \begin{bmatrix} r_{t,i}\cos\phi_i \\ r_{t,i}\sin\phi_i \end{bmatrix}$$

$$\mathrm{P}_t = \{\mathbf{p}_{t,i}\}_{i=1}^{N_t}$$

The robot evolves according to the differential-drive kinematic model:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \begin{bmatrix} v_t\cos\theta_t \\ v_t\sin\theta_t \\ \omega_t \end{bmatrix}\Delta t + \mathbf{w}_t$$

where $\mathbf{w}_t$ represents process noise.

Encoder + IMU Odometry

Each encoder tick corresponds to 0.0022m travel.
Right wheel distance:

$$d_R = \frac{FR + RR}{2} \cdot 0.0022$$

Left wheel distance:

$$d_L = \frac{FL + RL}{2} \cdot 0.0022$$

Linear displacement:

$$d = \frac{d_R + d_L}{2}$$

Angular velocity from IMU:

$$\omega_t = \text{IMU yaw rate}$$

Update

$$\theta_{t+1} = \theta_t + \omega_t \Delta t$$
$$x_{t+1} = x_t + d\cos\theta_{t+1}$$
$$y_{t+1} = y_t + d\sin\theta_{t+1}$$

This produces:

$$T_{t,t+1}^{odom} = \begin{bmatrix} R(\Delta\theta) & \mathbf{t} \\ 0 & 1 \end{bmatrix}$$

This odometry estimate is used as ICP initialization:

$$T^{init} = T_{t,t+1}^{odom}$$

Scan Matching

Read Point cloud from lidar:

$$P_t = \{\mathbf{p}_{t,i}\}_{i=1}^{N_t}$$
$$P_{t+1} = \{\mathbf{p}_{t+1,i}\}_{i=1}^{N_{t+1}}$$

Run ICP algorithm

$$T_{t,t+1}^{icp} = \text{ICP}(P_t, P_{t+1}, T^{init})$$

Get rectified Pose:

$$T_{t+1}^{rect} = T_{t,t+1}^{icp} T_t$$

Occupancy grid map

Given rectified trajectory and heading.

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}, \quad \theta_k$$

$$\mathbf{R}_k = \begin{bmatrix} \cos\theta_k & -\sin\theta_k \\ \sin\theta_k & \cos\theta_k \end{bmatrix}$$

$$\mathbf{p}_{k,i}^{world} = \mathbf{R}_k \mathbf{p}_{k,i} + \begin{bmatrix} x_k \\ y_k \end{bmatrix}$$

Map is consist of grid

$$\mathcal{M} = \{m_j\}_{j=1}^{M}$$

We focus on log-odds:

$$\ell_j = \log \frac{p(m_j = 1)}{1 - p(m_j = 1)}$$

从机器人位置到终点栅格(不包括终点)经过的所有栅格记为自由

$$\ell_j \leftarrow \ell_j + \ell_{free}$$

终点栅格记为占据:

$$\ell_j \leftarrow \ell_j + \ell_{occ}$$

In the end, color the map according to probability

$$p(m_j = 1) = \frac{1}{1 + e^{-\ell_j}}$$

$$\mathcal{M}_{occ} = \{p(m_j = 1)\}_{j=1}^{M}$$

Texture Mapping

Let d be the values at pixel (i, j) of the disparity image.

$$dd = (-0.00304d + 3.31)$$

Depth of the associated RGB color

$$z = \frac{1.03}{dd}$$

pixel location of the associated RGB color

$$\mathrm{rgbi} = (526.37i + 19276 - 7877.07dd)/585.051$$

$$\mathrm{rgbj} = (526.37j + 16662)/585.051$$

After the procedure above, we get 3D position and color of each pixel:

$$\mathbf{p}_i^c \in \mathbb{R}^3$$

$$\mathbf{c}_i = (R_i, G_i, B_i)$$

According to the mechanical design:

$$\mathbf{t}_r^c = \begin{bmatrix} 0.18 \\ 0.005 \\ 0.36 \end{bmatrix}$$

$$\mathbf{R}_{rpy} = \mathbf{R}_x(0.0)\mathbf{R}_y(0.36)\mathbf{R}_z(0.021)$$

$${}_c\mathbf{R}_r = \mathbf{R}_{rpy}\mathbf{R}_{base}$$

$\mathbf{R}_{base}$ is introduced for different setting of camera and robot bases
Camera to Robot:

$$\mathbf{p}_i^r = {}_c\mathbf{R}_r \mathbf{p}_i^c + {}_c\mathbf{t}_r$$

Robot to World.

$$\mathbf{p}_i^w = {}_r\mathbf{R}_w \mathbf{p}_i^r + {}_r\mathbf{t}_w$$

We only care about plane coordinates:

$$\mathbf{q}_i = \begin{bmatrix} x_i^w \\ y_i^w \end{bmatrix}$$

Get it's coordinates on the map

$$m_i = \left\lfloor \frac{\mathbf{q}_i - \mathbf{q}_{min}}{\text{resolution}} \right\rfloor$$

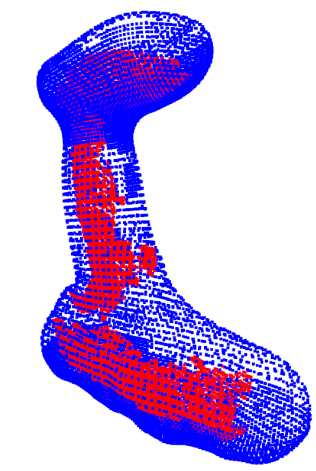
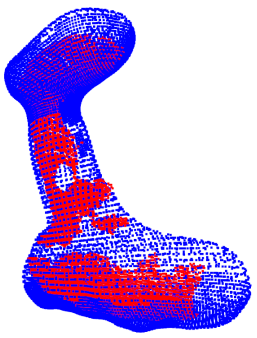
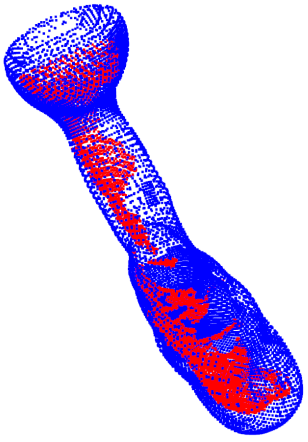
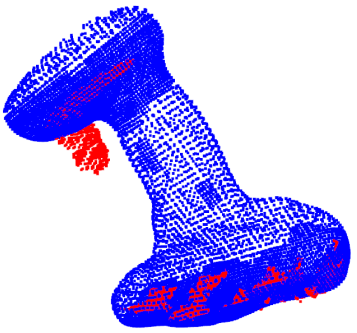
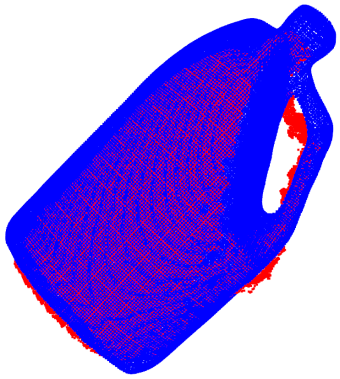
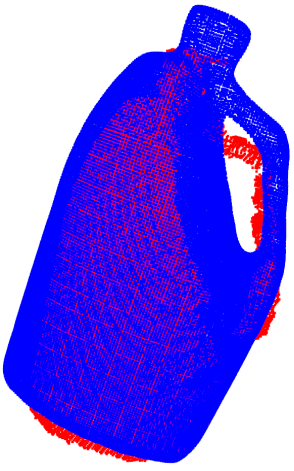
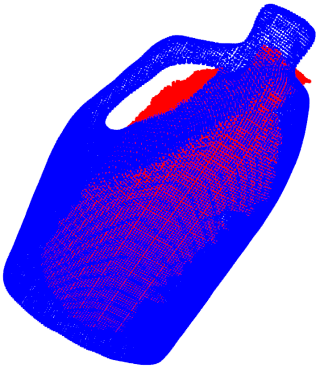
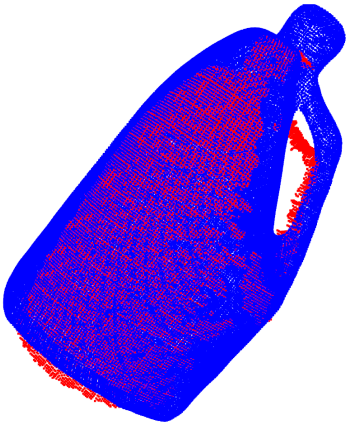
$$\mathcal{M}_{tex}(m_i) = \mathbf{c}_i$$

Pose graph optimization

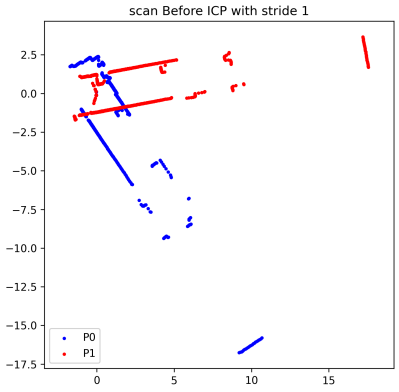
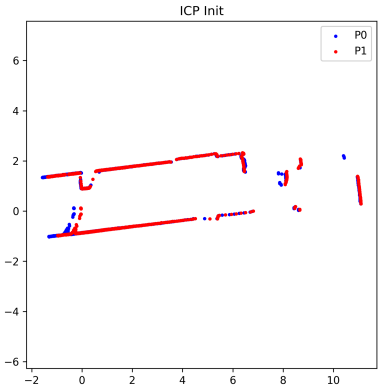
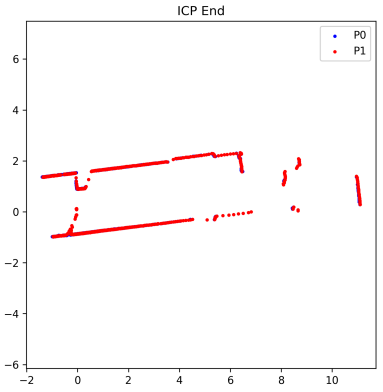
Odometry will give us a trajectroy but it will not recognize same place, in this part I will use scan matching to find loop closure and use it to optimize the location and orientation of robot.
The idea is simple, find two scan that are very similar and recognize them as the same position.

Result

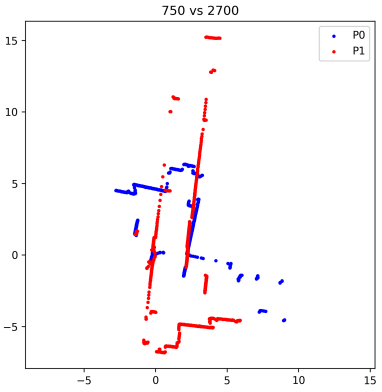
Validation of ICP

1	2	3	4
			
			

ICP works relatively well.

Before ICP	ICP Init	ICP End
		

Faulure cases

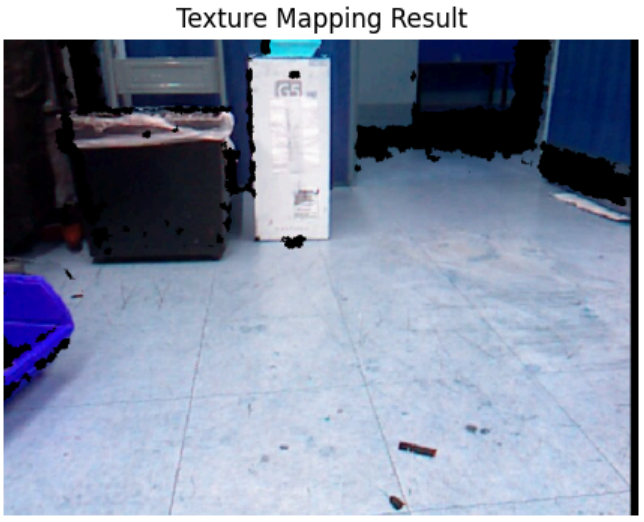


Human can easily recognize this but ICP frequently mismatch this since the range is quite huge.

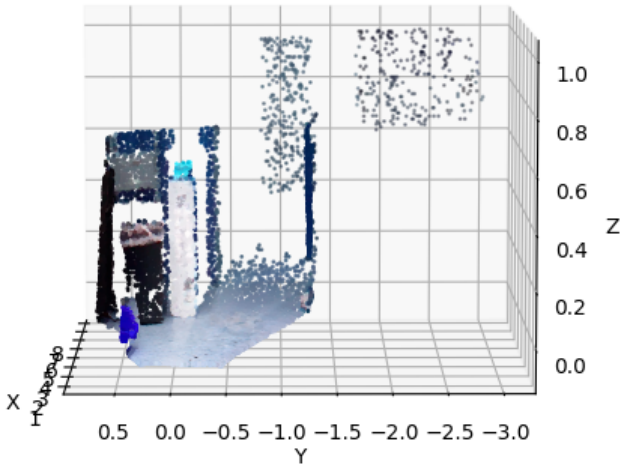
Validation of Camera

Map RGB to disparity image

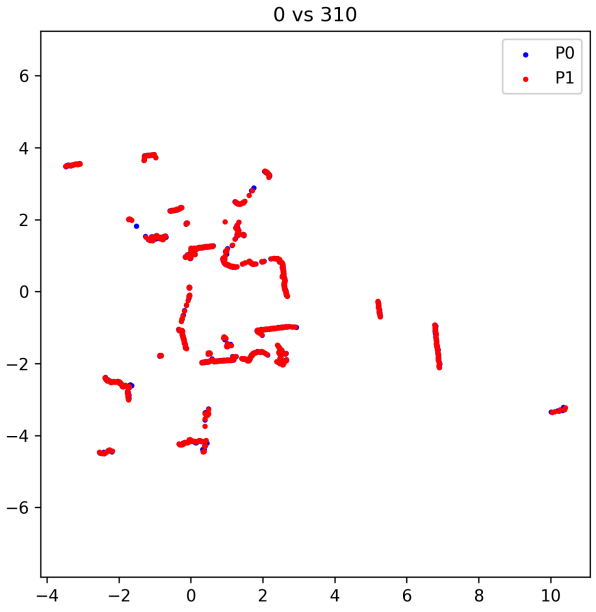


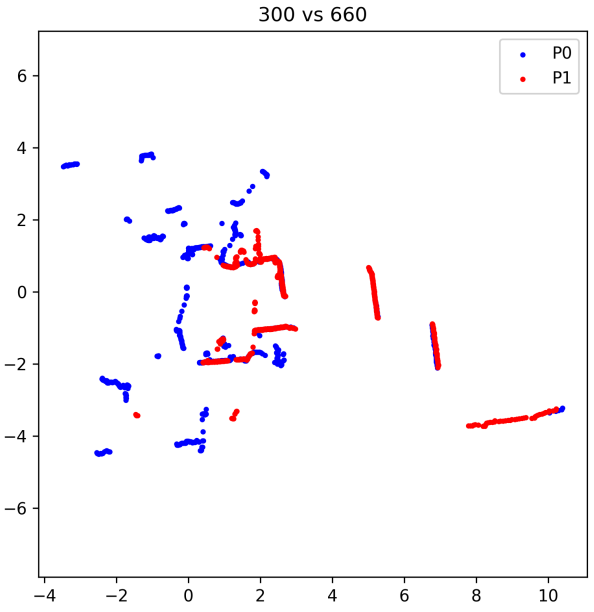


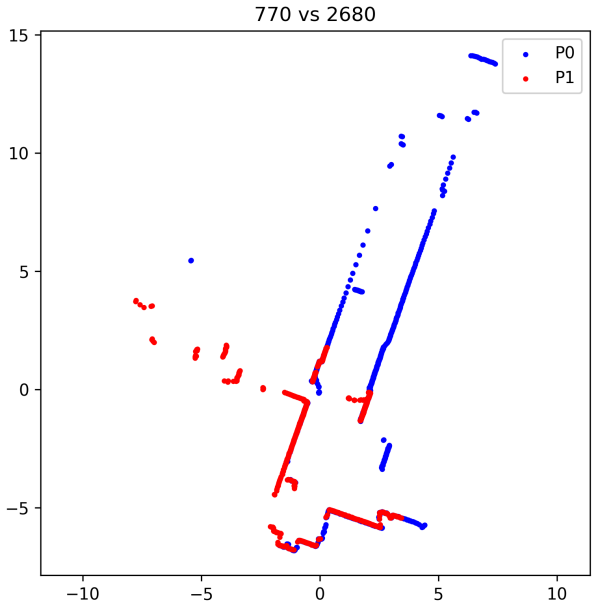
Reconstructed Points in world frame

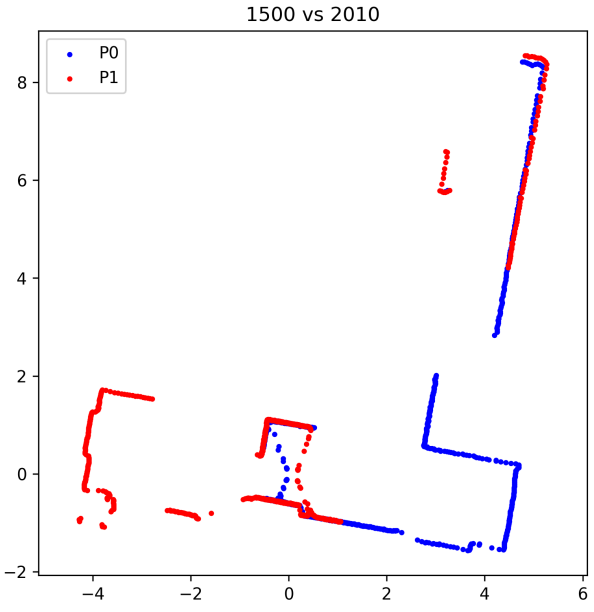


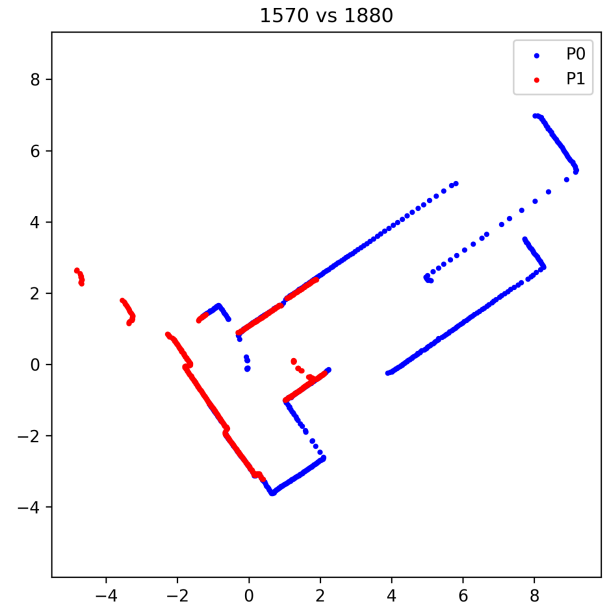
Loop Closure Detection

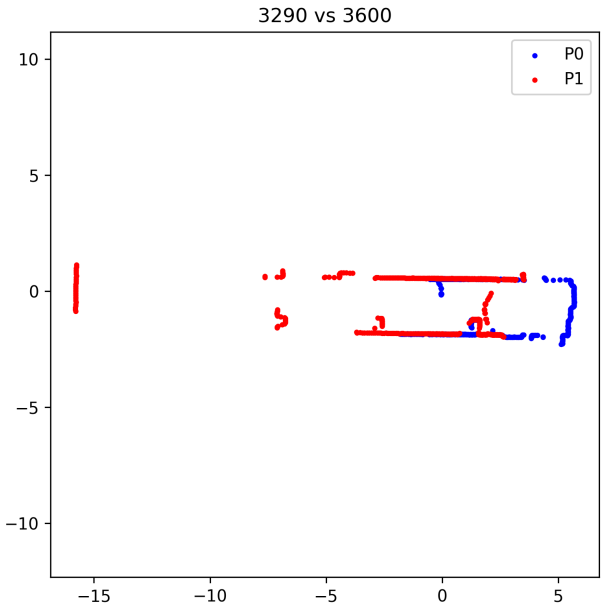


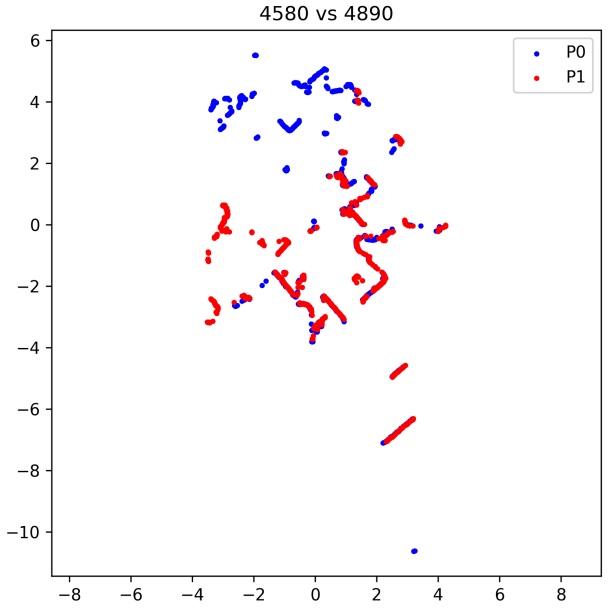


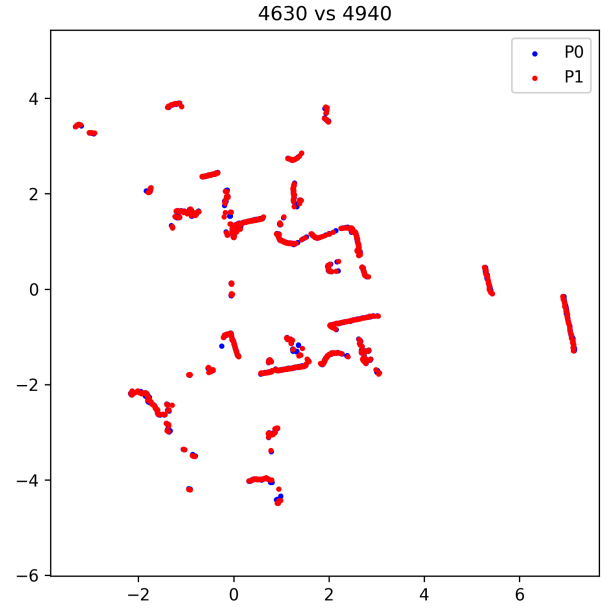


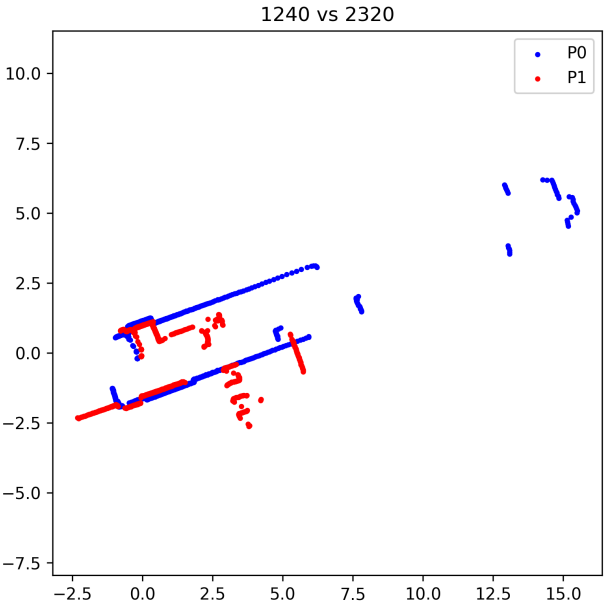


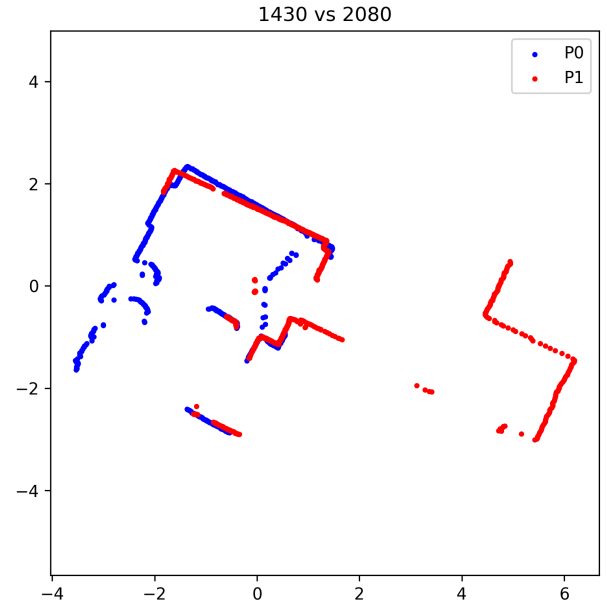

1570 vs 1880
P0
P1


3290 vs 3600
P0
P1

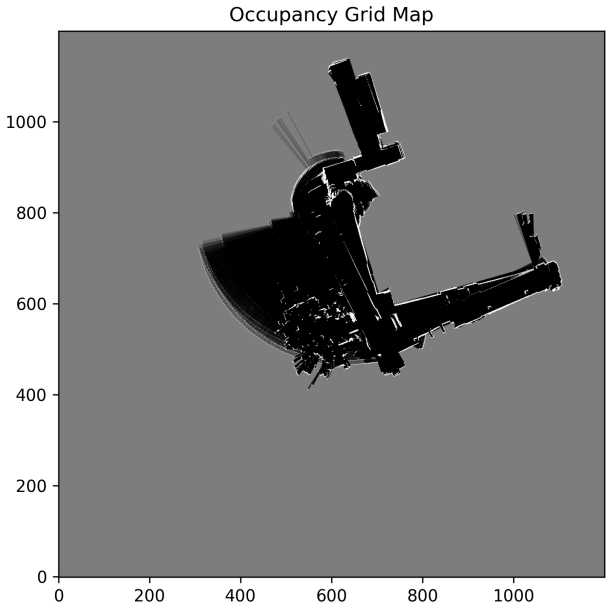

4580 vs 4890
P0
P1

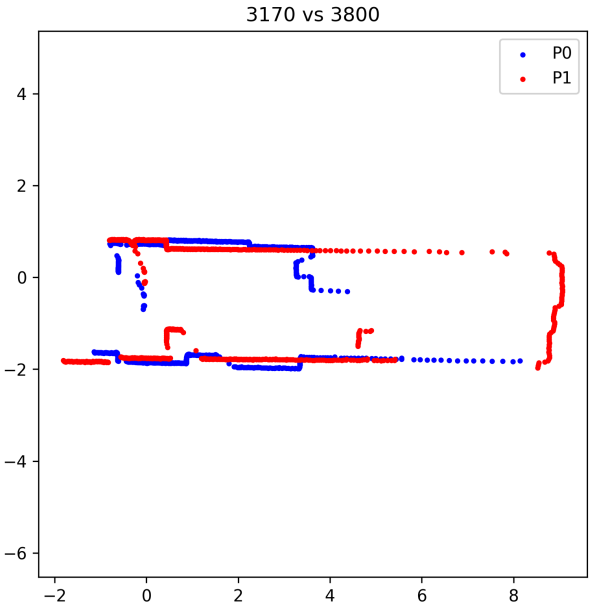

4630 vs 4940
P0
P1


1240 vs 2320
P0
P1


1430 vs 2080
P0
P1

Faulure Cases

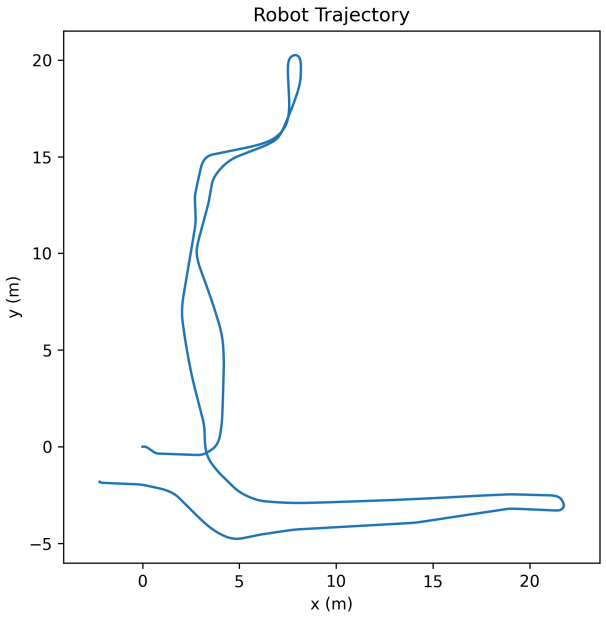
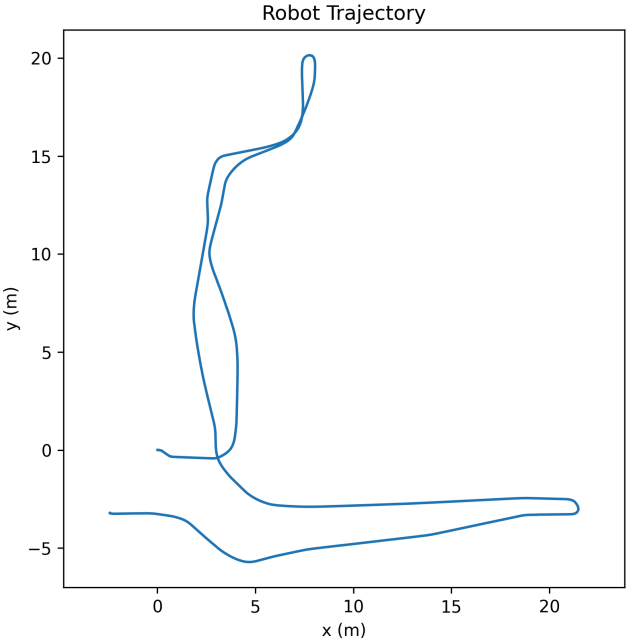




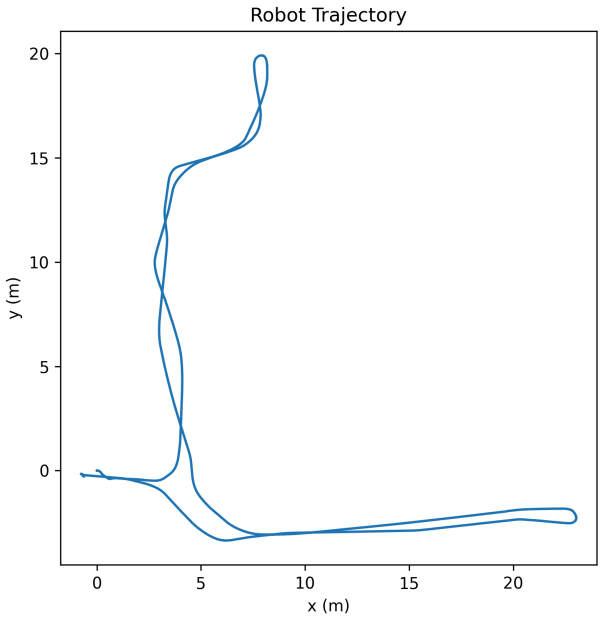
When robot met 2 parallel wall, it may recognize flipped orientation, and the localization may shift either.

Trajectory

20

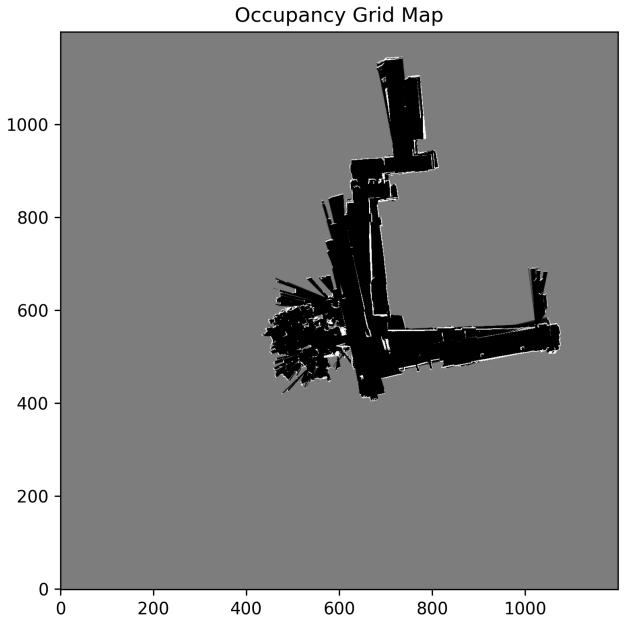
Odometry	Odometry+Lidar
	
Loop closure + pose graph optimization	

Odometry

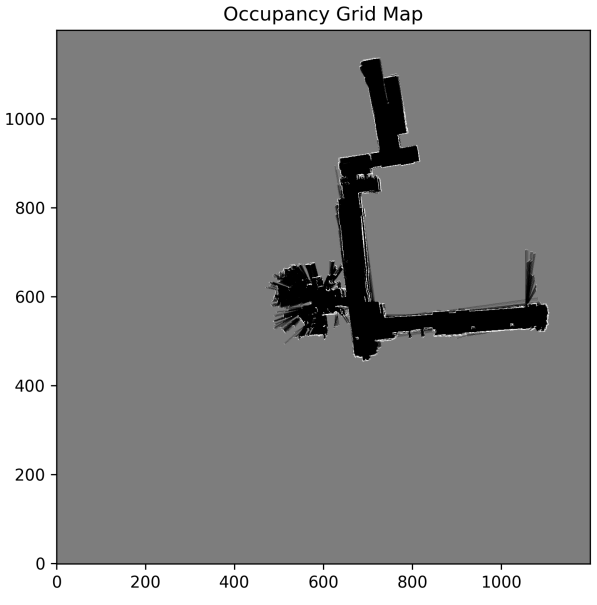


Odometry+Lidar

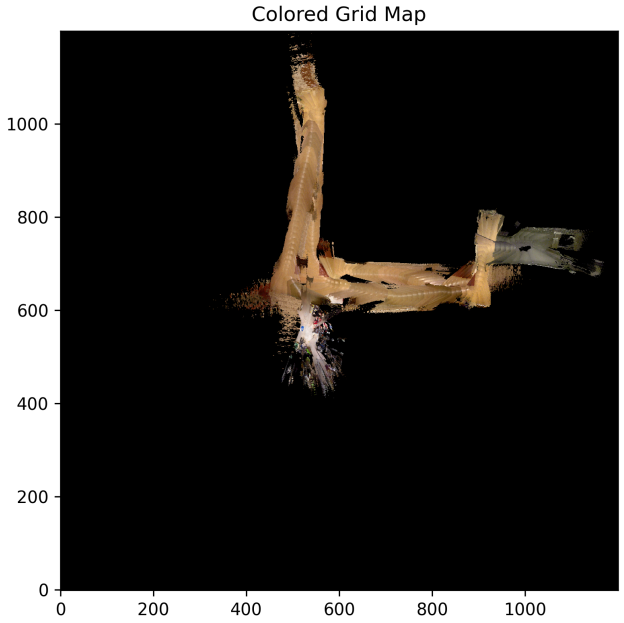
Odometry+Lidar



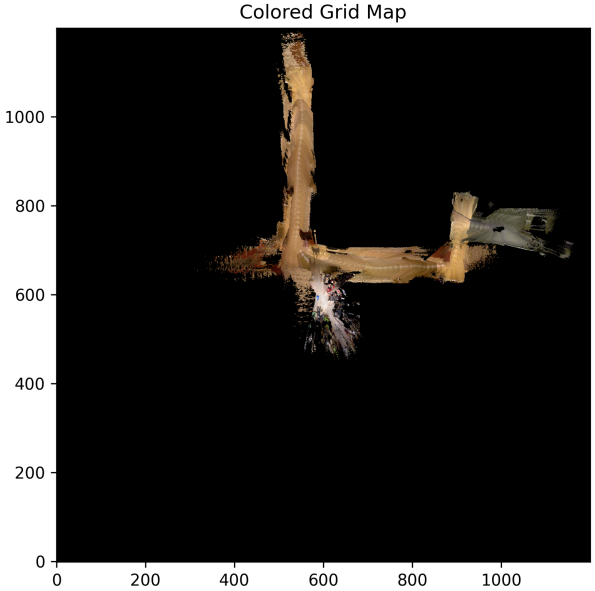
Loop closure + pose graph optimization



Odometry+Lidar

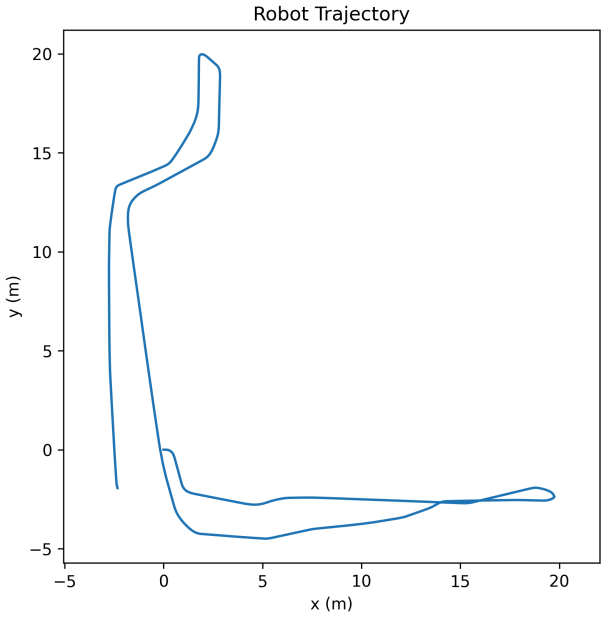


Loop closure + pose graph optimization

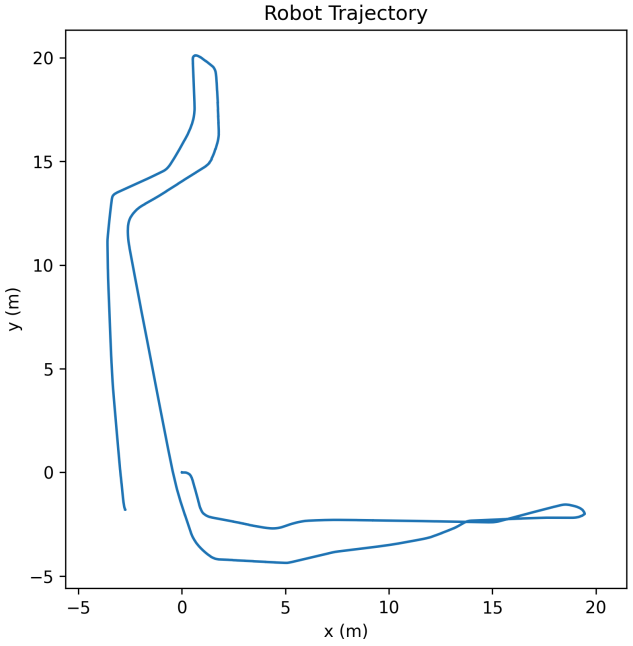


21

Odometry

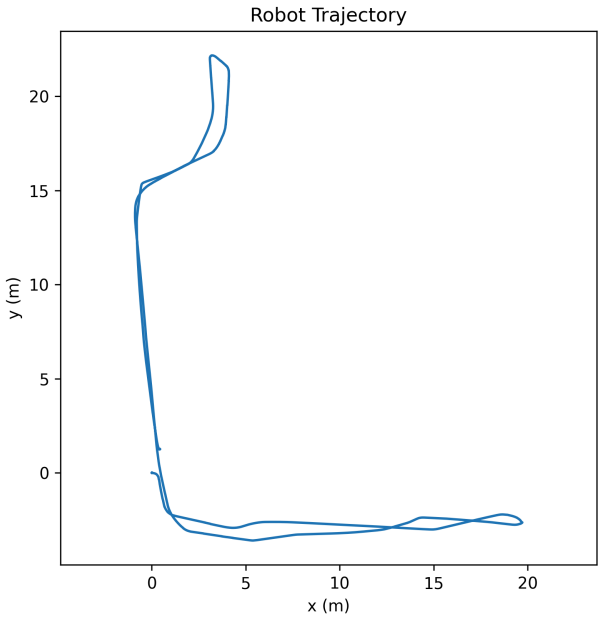


Odometry+Lidar



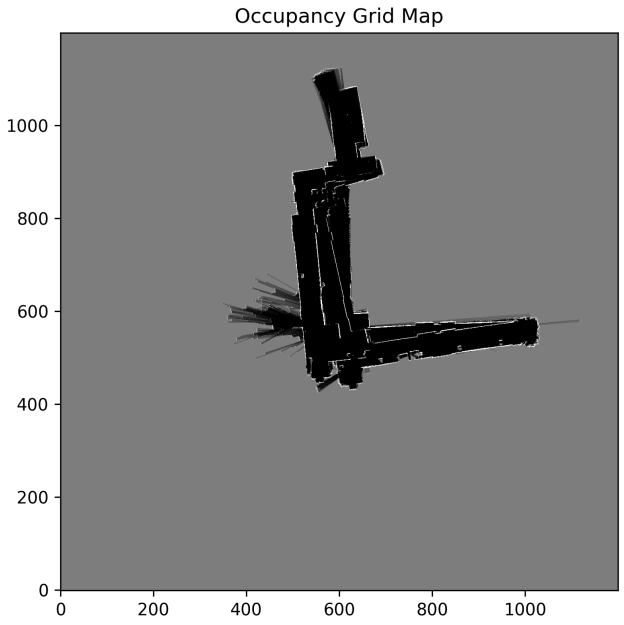
Loop closure + pose graph optimization

Odometry

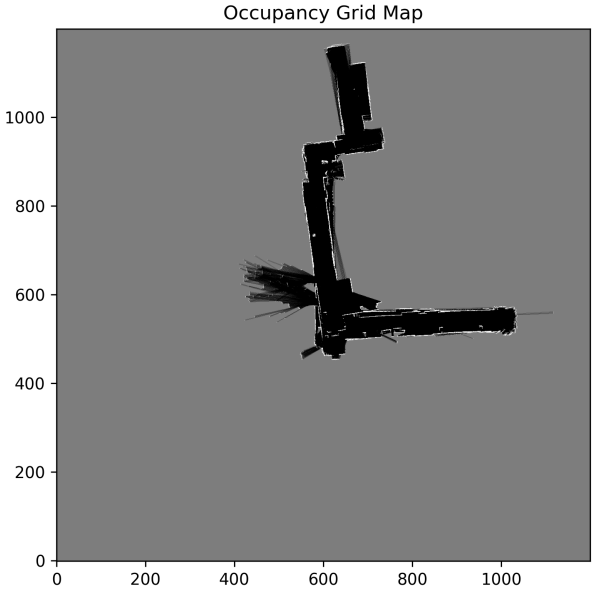


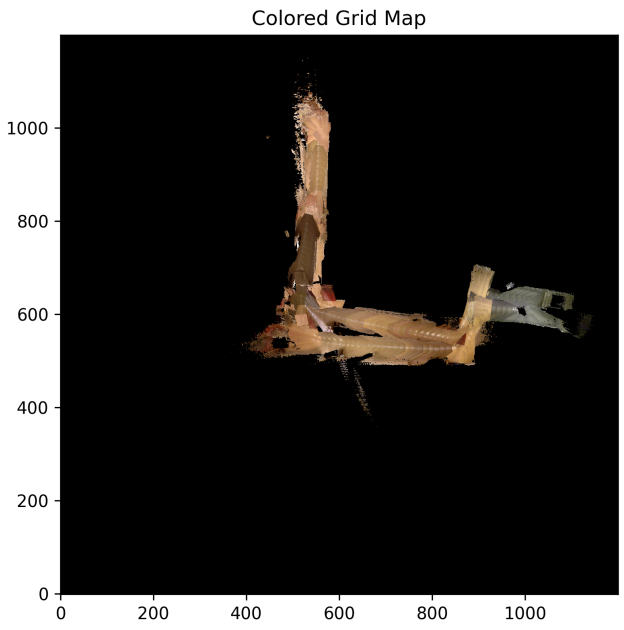
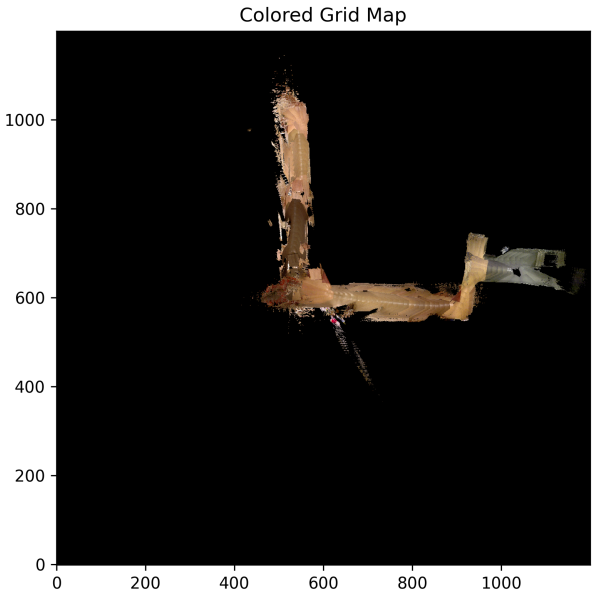
Odometry+Lidar

Odometry+Lidar



Loop closure + pose graph optimization



Odometry+Lidar	Loop closure + pose graph optimization
	

Loop closure works well, robot successfully find it and there is crosspoint on the trajectory
The mapping performance is much better